In IVD, we first clusters the points & then searches around the close ones. Don't search on the ones that are very far in another cluster.

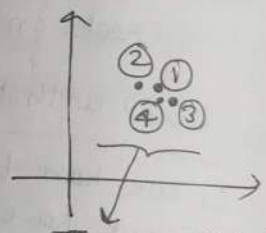
In HNSW, it is something very similar, we start with few points, search there & then stand adding more points & more points but using the info of what you had before. It's a much more effective way of searching. It works in ~~most~~ many-2 languages & cohere multilingual model is very useful for that.

[check cohere playground "Embed" tab & use many similar sentences with many languages]

↳ Is embedding + similarity enough?

Potential Prob^ns

Where was the last world cup? → ①
The previous world cup was in Russia → ②
The last world cup was in Qatar → ③
The world cup is in the moon → ④



These are all similar sentences because embedding only takes care of sentences that are similar. It does not really check if the actual answer was given.
[① & ③ are closed but ① & ② are also very closed] but ② is not the answer.

↳ So, looking for the closest sentence doesn't necessarily give us the answer. It gives us something similar to question.

How can we make search better?

Technique ①: Reranking.

Query: Where was the last world cup?

Responses
The world cup in 1986 was in Mexico
The world is round
The last world cup was in Qatar
The teacup ride is in Disney world

Say these are the similar sentences. & some of them are answers & some of them are non-answers.

So, we have a different model that ranks them to a model that is actually trained to see how good is a pair question answer & that model select the Rank-1 sentence "The last world cup was in Qatar".

Response	Rank
The world cup in 1986 was in mexico	2
The world is round	3
The last world cup was in Qatar	1
The teacup ride is in Disney world	4

So, How does this work?

Questions	Answers
Where was the last worldcup?	The last worldcup was in Qatar
What color is the sky?	The sky is blue
Where does the bear live?	The bear lives in the woods
What is an apple?	An apple is a fruit.

→ Model (Training) → High Scores.

Questions	Bad Answers
Where was the last world cup?	The last world cup was last year
What color is the sky?	The sky is ~~bad~~ red
Where does the bear live	The bear lives in NYC
What is an apple?	An apple is a vegetable

→ Model (Training) → Low Scores

↳ So, we're going to use an embedding + re-ranking to be able to search better.

Technique ②

Positive pai

Questions	An
Where was the last world cup?	Th cu Qa
What color is the sky?	Th
Where does the bear live?	Th i
What is an apple?	

↳ In positiv to the ex

↳ In nega to the

⇒ What is

Before seman keyword searc which are the keyword sear no. of word

But this what can stop words methods non-relev

Technique ② Positive & Negative pairs

Positive pairs

Questions	Answers
Where was the last world cup?	The last world cup was in Qatar
What color is the sky?	The sky is blue.
Where does the bear live?	The bear lives in the woods
What is an apple?	An apple is a fruit.

Negative pairs

Questions	Bad Answers
Where was the last worldcup?	The last world cup was last year
What color is the sky?	The sky is red
Where does the bear live?	The bear lives in NYC
What is an apple?	An apple is a vegetable

(Bad answers are closed but not exact answers)

↳ In positive pairs, we move close to the embedding of answers to the embedding of questions.

↳ In negative pairs, we move far to the " " " to the embedding of questions.

⇒ What is Not Semantic Search?

Before semantic search, the most popular way of searching was keyword search. Imagine that you have a list of many sentences, which are the responses. When you ask a question (query), keyword search looks for the sentence (response) with the largest no. of words in common with the query.

But this is not always give the correct response.

What can we do? We can improve keyword search, by removing stop words such as "the", "and", "is" etc. We can also use methods like TF-IDF in order to tell apart relevant from non-relevant words. However, there will always be the case.

in which, due to ambiguity of the language, synonyms & other roadblocks, keyword search will fail to find best response. So, we use

Semantic Search.

↳ In short, semantic search works as follows:

- It uses a text embedding to turn words into vectors.
- Uses similarity to find the vector among the responses which is the most similar to the vector corresponding to query.
- Outputs the response corresponding to this most similar vector.

↳ How to Search using Text Embeddings?

An embedding is a way to assign to each sentence (or more generally, to each text fragment, which can be as short as a word or as long as a full article), a vector, which is a list of numbers. A very important property of embeddings is the similar pieces of text get assigned to similar lists of numbers. For ex., the sentence "Hello, how are you?" & the sentence "Hi, what's up?" will be assigned list of similar numbers whereas the sentence "Tomorrow is Friday" will be assigned a list of numbers that are quite different from the two previous ones.

↳ Each query is closest to its corresponding response. That means if we used sementic search to search for the response to each of these 4 queries, we would get the correct response.

↳ However, here's a caveat. It is not the ideal way to compare pieces of text using Euclidean distance. The way that is most commonly used & has given the best results is similarity.

Using Similarity to Find the Best Document

Similarity is a way to tell if two pieces of text are similar or different. This uses text embeddings.

Let's say that similarity is a number assigned to each pair of documents, with the following properties:

- The similarity b/w a piece of text & itself is a very high number
- " " " two very similar pieces of text is a " "
- " " " " different " " " " small "

Here we use cosine similarity which has the extra property that the values it returns are b/w 0 & 1.

In order to perform semantic search, all you have to do is calculate the similarity b/w query & every pair of sentences & return the sentence with the highest similarity.

There is a hidden algo^m here that we did not mention, but it is very important: The nearest neighbors algorithm. In short, this algo^m finds the nearest neighbor of a point in a dataset. In this case, the algo^m found the nearest neighbor of the sentence "What is an apple?" & the response was the sentence "An apple is a fruit".

the language,
keyword search
So, we use

ks as follows:
words into vectors.
mong the responses
tor corresponding to query.
this most similar vector

sentence (or more
be as short as
ctor, which is a
of embeddings
similar list
ow are you?" &
gned list of
morrow is
rs that are
.
onse. That
he response
ect response.
to compare
s most
ity

Using Similarity to Find the Best Document

Similarity is a way to tell if two pieces of text are similar or different. This uses text embeddings.

↳ Let's say that similarity is a number assigned to each pair of documents, with the following properties:

- The similarity b/w a piece of text & itself is a very high number
- " " " two very similar pieces of text is a " " "
- " " " " different " " " " small "

↳ Here we use cosine similarity which has the extra property that the values it returns are b/w 0 & 1.

↳ In order to perform semantic search, all you have to do is calculate the similarity b/w query & every pair of sentences & return the sentence with the highest similarity.

↳ There is a hidden algo^m here that we did not mention, but is very important: The nearest neighbors algorithm. In short, this algo^m finds the nearest neighbor of a point in a dataset. In this case, the algo^m found the nearest neighbor of the sentence "What is an apple?" & the response was the sentence "An apple is a fruit".

↳ Nearest Neighbors - Pros & Cons & How to fix them

Nearest neighbors is a very simple & useful algo^m normally used for classificat^n. More generally, it's called K-nearest neighbors (KNN), where K is any number. If the task at hand is to classify a sentence as happy or sad (sentiment analysis), what 3-nearest neighbors would do is to look at the 3 closest neighbors to the sentence (in some embedding), & see if their majority (2) are happy or sad. That's the label it assigns to the sentence.

As you can see, KNN is exactly what we've been doing for semantic search here. Given a query, you look for the closest neighbor in the embedding & that's the response to the query. In the current examples, that method has worked well. However, KNN is not the fastest algo^m. The reason for this is that in order to find the neighbors of one point, one needs to calculate the distances b/w that point & all the other points in the dataset & then find the smallest one.

When dealing with massive archives, however, we can optimize for performance by slightly adjusting the algo^m to become approximate KNN. In particular, in search, there are several improvements that can speed up this process quite a lot. Here are 2 of them:

- Inverted File Index (IVF): Consists of clustering similar docs, then searching in clusters that are closest to the query.
- Hierarchical Navigable Small World (HNSW): Consists on starting with a few points & searching there. Then adding more points at each iteration & searching in each new space.

↳ Multilingual Sea

As you may have n
depends on the st
of the embedding
semantic search.
piece of text in
(~~leg~~ with length
to similar vectors
any language, o
all the other

↳ Are Embeddings

Consider the fo

Query: "Where

Responses:

1. The u
2. The u
3. The

↳ When we lo
all close, as ex
query is not
This response
correct statem
not the ans
cup is in the
semantically Cle
answers like
search.

How to fix them.

normally used
nearest neighbors
k at hand is
nent analysis).
k at the
e embedding).
. That's the

doing for
on the closest
nse to the
as worked well
n for this
ne point,
int & all
e smallest

e can optimize
on to become
e are several
ite a lot.

similar docs,
query.
sists on
adding
ew space.

Multilingual Search.

As you may have noticed, the performance of semantic search depends on the strength of embedding. Therefore, any superpowers of the embedding may translate ~~to~~ into superpowers of semantic search. In Cohere, the embedding will send any piece of text in any of these languages, to a vector (~~log~~ with length 768). Similar pieces of text will be sent to similar vectors. Therefore, one can search using a query in any language, and the model will search for answers in all the other languages.

Are Embeddings & Similarity Enough? (No)

Consider the following sentences.

Query: "Where is the world cup?"

Responses:

1. The world cup is in Qatar
2. The world cup is in moon
3. The previous world cup was in Russia

↳ When we locate them in the embedding above, they are all close, as expected. However, the closest sentence to the query is not response 1 (The correct answer), but response 3. This response ("The previous world cup is in Russia") is a correct statement & semantically close to the question but it's not the answer to the question. Response 2 ("The world cup is in the moon") is a completely wrong answer, but also semantically close to the query. So, It is likely the bogus answers like these will be the top results using semantic search.